Microcontroller Lab 5:

Objectives: Driving a stepper motor using assembly language, with and without msp4340.h to reference SFRs and control words.

Compare the size of code and memory utilization with the C program to run the stepper motor

Sequencing the phases indicated by 4 LEDs on the driver board, with the correct intervals so that the stepper motor completes one rotation in 1 minute

Things to do:

1. Refer to the manual/literature attached with reference to the LED blink task. **You can also open the example file of the LED blink assembly in CCS**.
2. Write the complete code for the task in the record. This must be done before coming to the lab
3. Comments for the lines the lines of code and understand what each line means
4. Write the calculations necessary to compute the pulse rate to the phases of the stepper motor. This has to demonstrated on the timings of LED blink
5. In the lab, you will also have to demonstrate the outputs on hardware for evaluation.
6. Compare the code size for this application in assembly and C.

Questions to be answered for lab 3. **You will not be allowed to conduct the experiment if you do not answer these questions and the ones in the reference section, and do not have the task code written in the record**

1. To which pins on the micro-controller are the motor drive outputs connected? What are the colours?
2. What is the principle used to generate the time delays required? What are the advantages and disadvantages?
3. How is the duration of phase pulses/LED timing computed from stepper motor datasheet? Show the calculations

Outputs to be shown.

1. Blinking 4 inbuilt LEDs in sequence at the correct rate. If a stepper motor is connected, it must complete one rotation in 1 minute
2. Memory map, register view and demonstrate single stepping showing affected registers and memory as applicable. You can use disassembly view if that helps.
3. Show the contents of the linker command file, msp430.h and SFRs that are modified
4. Run the assembly program by commenting out the references to register names and replacing them with the actual values the names represent

Resources required:

Microcontroller board, laptops for code development

Tabular column for results (part):

Address	**Contents**	**Code/ Data?**	**Core/Em?**	**# operands**	**Adr mode**	**SRC**	**DEST**	**Status Bits**	**Remarks**
0xC000	4031	Code	Core	2 operand	Immediate	0x400	Stack Pointer	------	MOV, W, R1,
0xC002	0400	Data							Immediate value
0xC004	4036								
0xC006	0120								
0xC008	40B6								
0xC010	5A80								

Reference:

Included below are the addresses of the SFRs mapped to the lower memory and the register that affects the watchdog timer. This is for reference in reading back from memory and deciphering the result of code execution. One of the objectives is to replace register names with the actual values they represent so that the need for msp430.h can be avoided with respect to code that uses these.

Q. Which of these are 8 bit and which are 16 bit registers?

Q. What is the value of watchdog password? What happens if any other value is used?

Q. What does a read on the password field return? Verify from memory map and code for this
Q. How is the watchdog disabled? What is the value to be written into the WDCTL to stop the watchdog?

Refer SLAS735J MSP430G2x53/MSP430G2x13 Manual from TI for complete description

MODULE	REGISTER DESCRIPTION	REGISTER NAME	OFFSET
Watchdog Timer+	Watchdog/timer control	WDTCTL	0120h
Port P2	Port P2 selection 2	P2SEL2	042h
	Port P2 resistor enable	P2REN	02Fh
	Port P2 selection	P2SEL	02Eh
	Port P2 interrupt enable	P2IE	02Dh
	Port P2 interrupt edge select	P2IES	02Ch
	Port P2 interrupt flag	P2IFG	02Bh
	Port P2 direction	P2DIR	02Ah
	Port P2 output	P2OUT	029h
	Port P2 input	P2IN	028h
Port P1	Port P1 selection 2	P1SEL2	041h
	Port P1 resistor enable	P1REN	027h
	Port P1 selection	P1SEL	026h
	Port P1 interrupt enable	P1IE	025h
	Port P1 interrupt edge select	P1IES	024h
	Port P1 interrupt flag	P1IFG	023h
	Port P1 direction	P1DIR	022h
	Port P1 output	P1OUT	021h
	Port P1 input	P1IN	020h
Special Function	SFR interrupt flag 2	IFG2	003h
	SFR interrupt flag 1	IFG1	002h
	SFR interrupt enable 2	IE2	001h
	SFR interrupt enable 1	IE1	000h

Refer SLAU144K MSP430F2xx, MSP430G2xx Family

Table 10-1. Watchdog Timer+ Registers

Address	Acronym	Register Name	Type	Reset	Section
120h	WDTCTL	Watchdog timer+ control	Read/write	6900h with PUC	Section 10.3.1
0h	IE1	SFR interrupt enable 1	Read/write	00h with PUC	Section 10.3.2
2h	IFG1	SFR interrupt flag 1	Read/write	00h with PUC	Section 10.3.3

Figure 10-2. WDTCTL Register

15	14	13	12	11	10	9	8
WDTPW							
rw-0	rw-1	rw-1	rw-0	rw-1	rw-0	rw-0	rw-1
7	6	5	4	3	2	1	0
WDTHOLD	WDTNMIES	WDTNMI	WDTTMSEL	WDTCNTCL	WDTSSEL	WDTISx	
rw-0	rw-0	rw-0	rw-0	r0(w)	rw-0	rw-0	rw-0

Table 10-2. WDTCTL Register Field Descriptions

Bit	Field	Type	Reset	Description
15-8	WDTPW	R/W	69h	Watchdog timer+ password. Always reads as 069h. Must be written as 05Ah. Writing any other value generates a PUC.
7	WDTHOLD	R/W	0h	Watchdog timer+ hold. This bit stops the watchdog timer+. Setting WDTHOLD = 1 when the WDT+ is not in use conserves power. 0b = Watchdog timer+ is not stopped 1b = Watchdog timer+ is stopped
6	WDTNMIES	R/W	0h	Watchdog timer+ NMI edge select. This bit selects the interrupt edge for the NMI interrupt when WDTNMI = 1. Modifying this bit can trigger an NMI. Modify this bit when WDTIE = 0 to avoid triggering an accidental NMI. 0b = NMI on rising edge 1b = NMI on falling edge
5	WDTNMI	R/W	0h	Watchdog timer+ NMI select. This bit selects the function for the $\overline{\text{RST}}$/NMI pin. 0b = Reset function 1b = NMI function
4	WDTTMSEL	R/W	0h	Watchdog timer+ mode select 0b = Watchdog mode 1b = Interval timer mode
3	WDTCNTCL	R/W	0h	Watchdog timer+ counter clear. Setting WDTCNTCL = 1 clears the count value to 0000h. WDTCNTCL is automatically reset. 0b = No action 1b = WDTCNT = 0000h
2	WDTSSEL	R/W	0h	Watchdog timer+ clock source select 0b = SMCLK 1b = ACLK
1-0	WDTISx	R/W	0h	Watchdog timer+ interval select. These bits select the watchdog timer+ interval to set the WDTIFG flag or generate a PUC. 00b = Watchdog clock source /32768 01b = Watchdog clock source /8192 10b = Watchdog clock source /512 11b = Watchdog clock source /64

Table 3-3. Source and Destination Operand Addressing Modes

As/Ad	Addressing Mode	Syntax	Description
00/0	Register mode	Rn	Register contents are operand
01/1	Indexed mode	X(Rn)	(Rn + X) points to the operand. X is stored in the next word.
01/1	Symbolic mode	ADDR	(PC + X) points to the operand. X is stored in the next word. Indexed mode X(PC) is used.
01/1	Absolute mode	&ADDR	The word following the instruction contains the absolute address. X is stored in the next word. Indexed mode X(SR) is used.
10/-	Indirect register mode	@Rn	Rn is used as a pointer to the operand.
11/-	Indirect autoincrement	@Rn+	Rn is used as a pointer to the operand. Rn is incremented afterwards by 1 for .B instructions and by 2 for .W instructions.
11/-	Immediate mode	#N	The word following the instruction contains the immediate constant N. Indirect autoincrement mode @PC+ is used.

Figure 3-9. Double Operand Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Op-code				S-Reg				Ad	B/W	As		D-Reg			

Figure 3-10. Single Operand Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Op-code									B/W	Ad		D/S-Reg			

Figure 3-11. Jump Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Op-code			C			10-Bit PC Offset									

	000	040	080	0C0	100	140	180	1C0	200	240	280	2C0	300	340	380	3C0
0xxx																
4xxx																
8xxx																
Cxxx																
1xxx	RRC	RRC.B	SWPB		RRA	RRA.B	SXT		PUSH	PUSH.B	CALL		RETI			
14xx																
18xx																
1Cxx																
20xx	JNE/JNZ															
24xx	JEQ/JZ															
28xx	JNC															
2Cxx	JC															
30xx	JN															
34xx	JGE															
38xx	JL															
3Cxx	JMP															
4xxx	MOV, MOV.B															
5xxx	ADD, ADD.B															
6xxx	ADDC, ADDC.B															
7xxx	SUBC, SUBC.B															
8xxx	SUB, SUB.B															
9xxx	CMP, CMP.B															
Axxx	DADD, DADD.B															
Bxxx	BIT, BIT.B															
Cxxx	BIC, BIC.B															
Dxxx	BIS, BIS.B															
Exxx	XOR, XOR.B															
Fxxx	AND, AND.B															

Figure 3-12. Core Instruction Map